F1 · DAY2 · WEEK1 MOIS 1 · SEMAINE 1 · JOUR 2 · FICHE 1 COURS AVANT PRATIQUE

Les bases de Python, depuis zéro

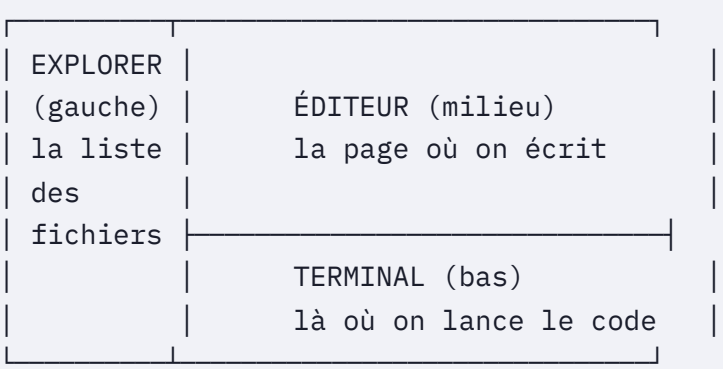
Tout ce qu'il faut savoir avant de toucher à un vrai projet : l'outil (VS Code), puis les 8 briques de base de Python. Chaque brique contient une explication simple, un exemple, le lien avec le métier d'AI/ML Engineer, un texte à dire à voix haute et un mini-exercice corrigé.

1. Le mode d'emploi de VS Code
2. Un programme, c'est quoi ?
3. Les variables
4. Les 4 types de base
5. Afficher et demander : print et input
6. Calculer et comparer
7. Décider : if, elif, else
8. Répéter : for et while
9. Les listes
10. Les fonctions
11. Lire une erreur
12. À savoir par cœur

> **Pourquoi on revient aux bases**
>
> Hier, on est allés trop vite : on a modifié un vrai projet avant de maîtriser l'outil et les briques de base. C'est comme apprendre à conduire directement sur l'autoroute. Aujourd'hui, on fait l'inverse : **d'abord comprendre, ensuite pratiquer**, sur de tout petits fichiers où rien ne peut casser.

0. Le mode d'emploi de VS Code

VS Code est un **cahier spécial pour écrire du code**. Il a 3 zones à connaître :



RACCOURCI	EFFET	À RETENIR
`Ctrl` + `S`	Enregistrer	Sans ça, Python lit l'ancienne version. Le ● sur l'onglet veut dire « pas enregistré ».
`Ctrl` + `Z`	Annuler	Rien n'est jamais perdu tant qu'on n'a pas enregistré.
`Ctrl` + `Y`	Refaire	Si on a annulé une fois de trop.
`Ctrl` + `ù`	Ouvrir / fermer le terminal	Le terminal est en bas.
`Ctrl` + `G`	Aller à une ligne	On tape le numéro, puis Entrée.
`Ctrl` + `Shift` + `K`	Supprimer la ligne entière	Plus sûr que de sélectionner à la souris.
`Inser`	Active / désactive le mode **OVR**	Si « OVR » s'affiche en bas, chaque lettre tapée **écrase** la suivante. Appuie sur `Inser` pour l'enlever.

> **La routine en 4 gestes, toujours la même**
>
> 1. **Écrire** le code au milieu.
> 2. **Enregistrer** avec `Ctrl` + `S`.
> 3. **Lancer** dans le terminal : `python nom_du_fichier.py`
> 4. **Lire** le résultat, ou l'erreur.

> **Fichier ou terminal ?**
>
> Dans le **fichier** `.py`, on écrit du **Python** (`print(...)`, `if ...`). Dans le **terminal**, on écrit des **commandes** pour l'ordinateur (`python fichier.py`, `cd`). Écrire `python fichier.py` dans le fichier provoque une `SyntaxError`.

---

1. Un programme, c'est quoi ?

Un programme est une **recette de cuisine pour l'ordinateur**. Python est le cuisinier. Il lit la recette **ligne par ligne, de haut en bas**, et il fait exactement ce qui est écrit. Ni plus, ni moins.

```
print("Étape 1")
print("Étape 2")     # toujours APRÈS l'étape 1
```

Python est **très précis et très bête** : une lettre en trop ou une parenthèse oubliée, et il s'arrête avec une erreur. Ce n'est pas un échec, c'est un message qui t'aide.

---

2. Les variables

> **Définition**
>
> Une **variable** est une **étiquette** collée sur une valeur, pour la retrouver plus tard grâce à son nom.

```
prenom = "Joseph"
age = 20
print(prenom)        # Joseph
age = age + 1        # on calcule à droite (21), PUIS on range à gauche
print(age)           # 21
```

Le signe `=` ne veut pas dire « égal ». Il veut dire « **range la valeur de droite dans l'étiquette de gauche** ».

Règles de nom : pas d'espace (on écrit `note_finale`), pas de chiffre au début, pas d'accent, des noms clairs (`moyenne` plutôt que `m`).

> **Dans le métier**
>
> Les réglages d'une IA sont des variables : `learning_rate = 0.001`, `epochs = 10`. Pour tester un autre réglage, on change une seule ligne.

> **À dire**
>
> « Une variable, c'est une étiquette sur une boîte. Le signe égal ne veut pas dire égal : il veut dire range ça là-dedans. »

Mini-exercice ★

Qu'affiche ce code ?

```
x = 5
x = x * 2
x = x + 1
print(x)
```

Corrigé : `11`. On a 5, puis 5 × 2 = 10, puis 10 + 1 = 11.

3. Les 4 types de base

TYPE	CE QUE C'EST	EXEMPLE	EN IA
`int`	nombre entier	`12`, `-3`	nombre d'images, d'époques
`float`	nombre à virgule (avec un **point**)	`0.92`, `3.5`	probabilités, scores
`str`	texte, entre guillemets	`"chat"`	noms, catégories
`bool`	vrai ou faux	`True`, `False`	« est-ce une fraude ? »

LE piège

`25` est un nombre. `"25"` est un **texte** qui ressemble à un nombre. On ne peut pas calculer avec : `"25" + 5` provoque une `TypeError`. Pour le transformer, on écrit `int("25")`, qui donne 25.

Dans le métier

Quand on lit un fichier de données, **tout arrive sous forme de texte.** Le premier travail du nettoyage consiste à convertir : `int(...)`, `float(...)`.

Mini-exercice ★

Quel est le type de `"3.5"`, de `3.5`, de `False` et de `7` ?

Corrigé : `str` (il y a des guillemets), `float`, `bool`, `int`. On peut le vérifier avec `print(type(...))`.

---

4. Afficher et demander : `print` et `input`

```
print("Bonjour")                        # affiche du texte
nom = input("Ton prénom ? ")            # pose une question, attend la réponse
print(f"Salut {nom} !")                 # f-string : on glisse une variable dans

age = int(input("Ton âge ? "))          # input renvoie TOUJOURS du texte → on co
print(f"Dans 10 ans : {age + 10} ans")
```

Le `f` devant les guillemets veut dire « **formaté** » : tout ce qui est entre `{ }` est remplacé par sa valeur.

Mini-exercice ★

Écrire un programme qui demande deux notes et affiche leur moyenne.

```
n1 = float(input("Note 1 ? "))
n2 = float(input("Note 2 ? "))
print(f"Moyenne : {(n1 + n2) / 2}")
```

5. Calculer et comparer

OPÉRATEUR	SENS	EXEMPLE	RÉSULTAT
`+ - * /`	les 4 opérations	`7 / 2`	`3.5`
`//`	division entière	`7 // 2`	`3`
`%`	reste de la division	`7 % 2`	`1`
`**`	puissance	`2 ** 3`	`8`
`== !=`	égal / différent	`5 == 5`	`True`
`< > <= >=`	plus petit / plus grand	`12 >= 10`	`True`

> **Piège**
>
> `=` sert à **ranger**, `==` sert à **comparer**. Écrire `if note = 10:` provoque une `SyntaxError`.

6. Décider : `if`, `elif`, `else`

Un programme peut **choisir** quoi faire, comme un aiguillage de train.

```
note = 14

if note >= 16:
    print("Très bien")
elif note >= 10:          # « sinon, si… »
    print("Admis")
else:                     # « dans tous les autres cas »
    print("Ajourné")
```

> **Les 2 règles d'or**
>
> 1. Toujours les **deux-points** `:` à la fin de la ligne `if`, `elif` ou `else`.
> 2. Les lignes « à l'intérieur » sont **décalées de 4 espaces** (on dit **indentées**). C'est le décalage qui dit à Python ce qui fait partie du `if`.

On combine des conditions avec `and` (les deux vraies), `or` (au moins une vraie) et `not` (l'inverse).

> **Dans le métier**
>
> Un modèle donne une probabilité, puis on décide : `if proba >= 0.5: print("fraude")`. Dans ExamPredict, `main.py` utilise exactement ça pour afficher HIGH, MEDIUM ou LOW.

Mini-exercice ★★

Avec `proba = 0.73`, afficher « sûr » si proba ≥ 0,9, « probable » si proba ≥ 0,5, sinon « peu probable ».

```
if proba >= 0.9:
    print("sûr")
elif proba >= 0.5:
    print("probable")
else:
    print("peu probable")      # affiche : probable
```

---

7. Répéter : `for` et `while`

`for` veut dire « **pour chaque élément**, fais ceci ». On l'utilise quand on sait sur quoi on boucle.

```
for note in [12, 15, 9]:
    print(note)              # 12, puis 15, puis 9

for i in range(3):           # range(3) donne 0, 1, 2
    print("Tour", i)
```

`while` veut dire « **tant que** c'est vrai, recommence ». On l'utilise quand on ne sait pas combien de tours il faudra.

```
essais = 0
while essais < 3:
    print("Essai", essais)
    essais = essais + 1      # sans cette ligne : boucle infinie !
```

> **Dans le métier**
>
> Entraîner une IA, c'est une boucle : `for epoch in range(10):` veut dire « relis toutes les données 10 fois et améliore-toi un peu à chaque tour ».

> **À dire**
>
> « Une IA apprend en boucle. À chaque tour, elle regarde ses erreurs et se corrige un peu. Le mot for, c'est le moteur de l'apprentissage. »

Mini-exercice ★★

Calculer la somme de `[12, 15, 9]` avec une boucle, sans utiliser `sum`.

```
total = 0
for n in [12, 15, 9]:
    total = total + n
print(total)                # 36
```

---

8. Les listes

Une liste est une **rangée de casiers numérotés. On compte à partir de 0.**

```
notes = [12, 15, 9]
notes[0]             # 12  (1er casier)
notes[-1]            # 9   (dernier casier)
len(notes)           # 3   (nombre de casiers)
notes.append(18)     # ajoute à la fin : [12, 15, 9, 18]
notes.remove(9)      # enlève la valeur 9 : [12, 15, 18]
15 in notes          # True  (est-ce présent ?)
sum(notes) / len(notes)   # la moyenne
```

Mini-exercice ★★

À partir de `notes = [12, 7, 15, 9, 18]`, afficher uniquement les notes supérieures ou égales à 10.

```
for n in notes:
    if n >= 10:
        print(n)              # 12, 15, 18
```

---

9. Les fonctions

Une fonction est un **mixeur**. On le construit une fois avec `def`, puis on peut l'utiliser autant de fois qu'on veut.

```
def moyenne(notes):                  # on CONSTRUIT le mixeur ; « notes » = l'ir
    total = sum(notes)
    return total / len(notes)        # ce qui SORT du mixeur

m = moyenne([12, 15, 9])             # on APPUIE sur le bouton
print(m)                             # 12.0
```

`return` ou `print` ?

`print` **montre** une valeur à l'écran, puis elle est perdue. `return` **rend** la valeur au programme, qui peut la ranger dans une variable et la réutiliser. Une fonction de calcul utilise `return`.

À dire

« Une fonction, c'est un mixeur. Avec def, tu le construis. Entre parenthèses, tu dis quels ingrédients il accepte. Avec return, tu dis ce qui en sort. Tant que tu n'appuies pas sur le bouton, il ne se passe rien. »

Mini-exercice ★★

Écrire une fonction `est_admis(note)` qui renvoie `True` si la note est au moins 10.

```
def est_admis(note):
    return note >= 10

print(est_admis(12))   # True
print(est_admis(8))    # False
```

10. Lire une erreur

Un message d'erreur se lit **de bas en haut** : la dernière ligne donne le **type** et l'**explication**, la ligne `line N` donne **l'endroit**.

ERREUR	EN FRANÇAIS	CAUSE FRÉQUENTE
`SyntaxError`	« ta phrase est mal écrite »	parenthèse, guillemet ou `:` oublié
`IndentationError`	« tes décalages sont faux »	il manque les 4 espaces sous un `if`, un `for` ou un `def`
`NameError`	« je ne connais pas ce mot »	faute de frappe, ou variable pas encore créée
`TypeError`	« tu mélanges des choses incompatibles »	`"25" + 5`
`ValueError`	« valeur impossible »	`int("bonjour")`
`IndexError`	« ce casier n'existe pas »	`notes[10]` sur une liste de 3 éléments
`FileNotFoundError`	« fichier introuvable »	commande lancée depuis le mauvais dossier

11. À savoir par cœur

1. La routine : écrire → `Ctrl` + `S` → `python fichier.py` → lire. Avec `Ctrl` + `Z`, rien n'est perdu.
2. Une variable est une étiquette. `=` range, `==` compare.
3. Il y a 4 types de base : `int`, `float`, `str` et `bool`. `"25"` n'est pas `25`.
4. `input` renvoie toujours du texte : on le convertit avec `int()` ou `float()`.
5. `if` / `elif` / `else` : deux-points à la fin, 4 espaces dessous.
6. `for` = pour chaque élément. `while` = tant que c'est vrai (attention à la boucle infinie).
7. Une liste se numérote à partir de 0 : `append` ajoute, `len` compte, `in` cherche.
8. `def` construit une fonction, l'appel l'utilise, `return` rend le résultat.
9. Une erreur se lit de bas en haut : le type, l'explication, puis la ligne.
